

Задача:

Реализовать модуль ядра chardev с именем int_stack.ko, а также пользовательскую утилиту kernel_stack. Также необходимо добавить USB-устройство в виде электронного ключа.

Выполнение работы и результаты:

1) Узнал два нужных id своей флешки, которую выбрал в качестве USB-устройства как электронного ключа.

Сначала сделал lsusb без:

```
namkitu@namkitu-IdeaPad:~/labs/lab5$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 002: ID 04f2:b6c2 Chicony Electronics Co., Ltd Integrated Camera
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 003 Device 002: ID 0bda:c123 Realtek Semiconductor Corp. Bluetooth Radio
Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
namkitu@namkitu-IdeaPad:~/labs/lab5$
```

А после опять вызвал lsusb уже подключив флешку:

```
namkitu@namkitu-IdeaPad:~/labs/lab5$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 002: ID 04f2:b6c2 Chicony Electronics Co., Ltd Integrated Camera
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 003 Device 002: ID 0bda:c123 Realtek Semiconductor Corp. Bluetooth Radio
Bus 003 Device 003: ID 1005:b113 Apacer Technology, Inc. Handy Steno/AH123 / Handy Steno 2.0/HT203
Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
namkitu@namkitu-IdeaPad:~/labs/lab5$
```

Так узнал, что нужные мне id это 1005 и b113

2) Почистил оставшиеся после 4й лабы ресурсы

```
namkitu@namkitu-IdeaPad:~/labs/lab5$ sudo rmmod int_stack
[sudo] password for namkitu:
Sorry, try again.
[sudo] password for namkitu:
namkitu@namkitu-IdeaPad:~/labs/lab5$ sudo rm /dev/int_stack
```

3) После этого скомпилировал int_stack.c и kernel_stack.c

```

namkitu@namkitu-IdeaPad:~/labs/lab5$ make
make -C /lib/modules/6.17.0-20-generic/build M=/home/namkitu/labs/lab5 modules
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-20-generic'
make[2]: Entering directory '/home/namkitu/labs/lab5'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04
.1) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
make[2]: Leaving directory '/home/namkitu/labs/lab5'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-20-generic'
gcc kernel_stack.c -o kernel_stack
namkitu@namkitu-IdeaPad:~/labs/lab5$

```

Код Makefile:

```

obj-m += int_stack.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all:
    make -C $(KDIR) M=$(PWD) modules
    gcc kernel_stack.c -o kernel_stack

clean:
    make -C $(KDIR) M=$(PWD) clean
    rm -f kernel_stack

```

4) Сначала проверил логи и не увидел, что стек создан при вставке флешки. После поисков проблемы понял, что нужно выключить драйвера

```

namkitu@namkitu-IdeaPad:~/labs/lab5$ sudo modprobe -r uas
namkitu@namkitu-IdeaPad:~/labs/lab5$ sudo modprobe -r usb_storage
namkitu@namkitu-IdeaPad:~/labs/lab5$

```

5) После этого в логах уже появилось упоминание, что int_stack создается при вставке флешки

```

namkitu@namkitu-IdeaPad:~/labs/lab5$ sudo dmesg | tail -n 20
[27824.077671] usb 3-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[27824.077679] usb 3-1: Product: USB FLASH DRIVE
[27824.077686] usb 3-1: Manufacturer:
[27824.077693] usb 3-1: SerialNumber: 07BB020203360017
[27824.078611] int_stack: USB Key inserted!
[27824.111221] usbcore: registered new interface driver usb-storage
[27824.117425] usbcore: registered new interface driver uas
[27862.718372] usb 3-1: USB disconnect, device number 7
[27862.718858] int_stack: USB Key removed!
[27864.768181] usbcore: deregistering interface driver uas
[27864.791302] usbcore: deregistering interface driver usb-storage
[27929.523822] usb 3-1: new high-speed USB device number 8 using xhci_hcd
[27929.656032] usb 3-1: New USB device found, idVendor=1005, idProduct=b113, bcd
Device= 1.00
[27929.656053] usb 3-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[27929.656061] usb 3-1: Product: USB FLASH DRIVE
[27929.656069] usb 3-1: Manufacturer:
[27929.656076] usb 3-1: SerialNumber: 07BB020203360017
[27929.657216] int_stack: USB Key inserted!
[27929.690265] usbcore: registered new interface driver usb-storage
[27929.694337] usbcore: registered new interface driver uas
namkitu@namkitu-IdeaPad:~/labs/lab5$

```

6) Провел простой тест и посмотрел, что /dev/int_stack появляется и исчезает как нужно

```

namkitu@namkitu-IdeaPad:~/labs/lab5$ ls -l /dev/int_stack
crw----- 1 root root 510, 0 May 13 01:34 /dev/int_stack
namkitu@namkitu-IdeaPad:~/labs/lab5$ ls -l /dev/int_stack
ls: cannot access '/dev/int_stack': No such file or directory
namkitu@namkitu-IdeaPad:~/labs/lab5$ ls -l /dev/int_stack
crw----- 1 root root 510, 0 May 13 01:34 /dev/int_stack
namkitu@namkitu-IdeaPad:~/labs/lab5$

```

7) Провел полноценные тесты стека, как делал в лабе 4, но в этот раз уже с возможностью достать флешку посреди этого теста

```

namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack set-size 2
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack push 1
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack push 2
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack push 3
ERROR: stack is full
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack pop
2
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack pop
1
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack pop
NULL
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack push 1
ERROR: USB key is not inserted
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack push 1
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack pop
ERROR: USB key is not inserted
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack pop
1
namkitu@namkitu-IdeaPad:~/labs/lab5$ █

```

Данный тест показывает, что:

- работает set-size
- выводится корректная ошибка про заполнение стека, как в ТЗ
- поп выводит нужные данные, а когда их нет NULL
- программа не работает без флешки
- данные не теряются, если сделать push, вытащить и вставить обратно флешку, а потом сделать поп

```

namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack set-size 3
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack push 1
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack push 2
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack push 3
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack push 4
ERROR: stack is full
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./
kernel_stack screens/
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack unwind
ERROR: USB key is not inserted
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack unwind
3
2
1
namkitu@namkitu-IdeaPad:~/labs/lab5$

```

Данный тест показывает, что работает команда unwind

8) В конце проверил, что код ошибки при вставке в заполненный стек действительно равен -ERANGE

```

namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack set-size 1
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack push 1
namkitu@namkitu-IdeaPad:~/labs/lab5$ ./kernel_stack push 2
ERROR: stack is full
namkitu@namkitu-IdeaPad:~/labs/lab5$ echo $?
222
namkitu@namkitu-IdeaPad:~/labs/lab5$

```

В данном случае вывод равен 222 или же 256 – 34

Программа int_stack была реализована на C

Исходный код:

```

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/slab.h>
#include <linux/mutex.h>
#include <linux/cdev.h>
#include <linux/usb.h>
#include <linux/device.h>

```

```

#define DEVICE_NAME "int_stack"
#define STACK_IOCTL_SET_SIZE _IOW('k', 1, int)
#define USB_MANUFACTURER_ID 0x1005
#define USB_ID 0xb113

```

```

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Kirill Samsonov");
MODULE_DESCRIPTION("A kernel-based integer stack character driver with USB
key");

```

```

struct int_stack_dev {
    int *buffer;
    int top;
    int max_size;
    struct mutex lock;
    struct cdev cdev;
    dev_t dev_num;
    struct class *cls;
};

```

```

static struct int_stack_dev *stack_dev;

```

```

static int stack_open(struct inode *inode, struct file *file) {
    pr_info("int_stack: Device opened\n");
    return 0;
}

```

```
}
```

```
static int stack_release(struct inode *inode, struct file *file) {  
    pr_info("int_stack: Device closed\n");  
    return 0;  
}
```

```
static ssize_t stack_read(struct file *file, char __user *buf, size_t count, loff_t *ppos)  
{  
    int val;  
    ssize_t ret = 0;  
    if (count < sizeof(int)) return -EINVAL;  
  
    mutex_lock(&stack_dev->lock);  
    if (stack_dev->top <= 0) {  
        ret = 0;  
    } else {  
        stack_dev->top--;  
        val = stack_dev->buffer[stack_dev->top];  
        if (copy_to_user(buf, &val, sizeof(int))) {  
            ret = -EFAULT;  
        } else {  
            ret = sizeof(int);  
        }  
    }  
    mutex_unlock(&stack_dev->lock);  
    return ret;  
}
```

```
static ssize_t stack_write(struct file *file, const char __user *buf, size_t count, loff_t  
*ppos) {  
    int val;  
    ssize_t ret = 0;  
    if (count < sizeof(int)) return -EINVAL;  
  
    mutex_lock(&stack_dev->lock);  
    if (stack_dev->top >= stack_dev->max_size) {  
        ret = -ERANGE;  
    } else {  
        if (copy_from_user(&val, buf, sizeof(int))) {  
            ret = -EFAULT;  
        } else {  
            stack_dev->buffer[stack_dev->top] = val;  
            stack_dev->top++;  
            ret = sizeof(int);  
        }  
    }  
}
```

```

    }
}
mutex_unlock(&stack_dev->lock);
return ret;
}

```

```

static long stack_ioctl(struct file *file, unsigned int cmd, unsigned long arg) {
    int new_size;
    int *new_buffer;
    if (cmd != STACK_IOCTL_SET_SIZE) return -ENOTTY;

    if (copy_from_user(&new_size, (int __user *)arg, sizeof(int))) return -EFAULT;
    if (new_size <= 0) return -EINVAL;

    mutex_lock(&stack_dev->lock);
    new_buffer = kmalloc_array(new_size, sizeof(int), GFP_KERNEL);
    if (!new_buffer) {
        mutex_unlock(&stack_dev->lock);
        return -ENOMEM;
    }
    kfree(stack_dev->buffer);
    stack_dev->buffer = new_buffer;
    stack_dev->max_size = new_size;
    stack_dev->top = 0;
    mutex_unlock(&stack_dev->lock);
    return 0;
}

```

```

static struct file_operations fops = {
    .owner = THIS_MODULE,
    .read = stack_read,
    .write = stack_write,
    .unlocked_ioctl = stack_ioctl,
    .open = stack_open,
    .release = stack_release,
};

```

```

static int usb_stack_probe(struct usb_interface *interface, const struct usb_device_id
*id) {
    int ret;
    pr_info("int_stack: USB Key inserted!\n");
    cdev_init(&stack_dev->cdev, &fops);
    ret = cdev_add(&stack_dev->cdev, stack_dev->dev_num, 1);
    if (ret < 0) return ret;
}

```

```

    stack_dev->cls = class_create("int_stack_class");
    if (IS_ERR(stack_dev->cls)) {
        cdev_del(&stack_dev->cdev);
        return PTR_ERR(stack_dev->cls);
    }

    device_create(stack_dev->cls, NULL, stack_dev->dev_num, NULL,
DEVICE_NAME);
    return 0;
}

static void usb_stack_disconnect(struct usb_interface *interface) {
    device_destroy(stack_dev->cls, stack_dev->dev_num);
    class_destroy(stack_dev->cls);
    cdev_del(&stack_dev->cdev);
    pr_info("int_stack: USB Key removed!\n");
}

static struct usb_device_id usb_stack_table[] = {
    { USB_DEVICE(USB_MANUFACTURER_ID, USB_ID) },
    { }
};
MODULE_DEVICE_TABLE(usb, usb_stack_table);

static struct usb_driver stack_usb_driver = {
    .name = "int_stack_usb_driver",
    .probe = usb_stack_probe,
    .disconnect = usb_stack_disconnect,
    .id_table = usb_stack_table,
};

static int __init int_stack_init(void) {
    int ret;
    stack_dev = kzalloc(sizeof(struct int_stack_dev), GFP_KERNEL);
    if (!stack_dev) return -ENOMEM;

    ret = alloc_chrdev_region(&stack_dev->dev_num, 0, 1, DEVICE_NAME);
    if (ret < 0) { kfree(stack_dev); return ret; }

    mutex_init(&stack_dev->lock);
    stack_dev->max_size = 10;
    stack_dev->buffer = kmalloc_array(stack_dev->max_size, sizeof(int),
GFP_KERNEL);

    ret = usb_register(&stack_usb_driver);

```

```

if (ret) {
    unregister_chrdev_region(stack_dev->dev_num, 1);
    if (stack_dev->buffer) kfree(stack_dev->buffer);
    kfree(stack_dev);
    return ret;
}
return 0;
}

```

```

static void __exit int_stack_exit(void) {
    usb_deregister(&stack_usb_driver);
    unregister_chrdev_region(stack_dev->dev_num, 1);
    if (stack_dev->buffer) kfree(stack_dev->buffer);
    kfree(stack_dev);
    pr_info("int_stack: Module unloaded\n");
}

```

```

module_init(int_stack_init);
module_exit(int_stack_exit);

```

Программа kernel_stack была тоже релизована на C
Исходный код:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <string.h>
#include <errno.h>

#define DEVICE_PATH "/dev/int_stack"
#define STACK_IOCTL_SET_SIZE _IOW('k', 1, int)

void handle_error(int code) {
    if (code == ERANGE || code == -ERANGE) {
        fprintf(stderr, "ERROR: stack is full\n");
        exit(-34);
    }
    perror("ERROR");
    exit(code);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <command> [arg]\n", argv[0]);
    }
}

```

```

    return 1;
}

int fd = open(DEVICE_PATH, O_RDWR);
if (fd < 0) {
    if (errno == ENOENT) {
        fprintf(stderr, "ERROR: USB key is not inserted\n");
    } else {
        perror("ERROR");
    }
    return 1;
}

if (strcmp(argv[1], "set-size") == 0) {
    if (argc < 3) return 1;
    int size = atoi(argv[2]);

    if (size <= 0) {
        fprintf(stderr, "ERROR: size should be > 0\n");
        close(fd);
        return 1;
    }

    if (ioctl(fd, STACK_IOCTL_SET_SIZE, &size) < 0) {
        handle_error(errno);
    }
}

else if (strcmp(argv[1], "push") == 0) {
    if (argc < 3) return 1;
    int val = atoi(argv[2]);

    if (write(fd, &val, sizeof(int)) < 0) {
        handle_error(errno);
    }
}

else if (strcmp(argv[1], "pop") == 0) {
    int val;
    ssize_t res = read(fd, &val, sizeof(int));

    if (res == 0) {
        printf("NULL\n");
    } else if (res < 0) {
        handle_error(errno);
    } else {
        printf("%d\n", val);
    }
}

```

```
    }  
}  
else if (strcmp(argv[1], "unwind") == 0) {  
    int val;  
    while (read(fd, &val, sizeof(int)) > 0) {  
        printf("%d\n", val);  
    }  
}  
  
close(fd);  
return 0;  
}
```